B. Security Requirements (Mandatory)

1. Input Validation (OWASP ASVS V5 / A1)

- Use whitelisting, regex, built-in validators.
- Prevent SQL injection with ORM / parameterized queries.
- Validate all client-side + server-side inputs.

2. Authentication & Session Management (OWASP ASVS V2 / A2)

- Secure login flow
- Strong password rules
- Session timeout
- CSRF protection enabled
- Use secure cookies (HttpOnly, Secure flag)

3. Access Control (OWASP ASVS V4 / A5)

- Enforce RBAC on all endpoints and API routes.
- No direct object reference (IDOR).

4. Error Handling (OWASP ASVS V7)

- No stack traces exposed to users
- Use custom error pages (400, 403, 404, 500)

5. Sensitive Data Protection (OWASP ASVS V3 / A6)

- Hash passwords using bcrypt/Argon2
- No plaintext logs for credentials or tokens
- Use TLS/HTTPS for all connections

6. File Upload Security

- Validate file type (MIME + extension)
- Restrict upload size
- Store uploaded files outside web root
- Rename files to random UUID

7. Configuration Security

- Use .env file for secrets
- Disable debug mode in production
- Update dependencies
- Secure API keys

8. Logging & Monitoring (OWASP ASVS V7)
    - Log failed logins, admin actions, and suspicious activities
    - Logging must not include sensitive data
    - Admin can view logs

9. Dependency Management
    - Update third-party libraries
    - Verify sources
    - Conduct dependency scanning (e.g., OWASP Dependency Check, Snyk)-CSA

10. Output Encoding / Escaping (OWASP ASVS V5 / A3)
    - Escape HTML output
    - Prevent XSS by using built-in template escaping mechanisms

4. Deliverables

A. Technical Report (Mandatory Sections)

1. Introduction
2. System Architecture + Data Flow Diagram with trust Boundries
3. Secure Coding Implementation
    - Explain how each OWASP item is applied
4. Security Testing
    - Manual Code Review Checklist (table included)
    - Static analysis results (Bandit, ESLint, SonarLint, etc.)
    - Dynamic testing (OWASP ZAP, Burp Suites)
5. Screenshots of Working Application
6. Conclusion

B. Source Code Submission

- Must include instructions to install and run
- Must include .env.example (not real secrets)
- Include dependency list (requirements.txt, package.json, etc.)- SCA

C. Security Checklist Evaluation

Students must evaluate their own project against the Manual Code Review Checklist provided:

- Input Validation
- Authentication
- Access Control
- Error Handling
- Sensitive Data Protection

- File Upload Security
- Configuration Security
- Logging & Monitoring
- Dependency Management
- Output Encoding

D. Video Presentation (10 mins each max , min = 4 mins) for each video

1 group compulsory to deliver more than 1 video

You may delegate the presentation for each team member

For example :

Member 1 : code generation /development(Functionlity Requirement CRUD)

Member 2 : Result testing may break into a few testing videos(attack included)

Member 3 : mitigation apply specifically focus on issues bug and flaw

Member 4 :CI /CD - managing Github as team repository (commit updates etc)

Cover:

1. Web Application demo
2. OWASP controls/mitigation implemented
3. Security test results
4. Discussion on preventing attacks
5. CI / CD task

6. Suggested Secure Frameworks

Students may choose one:

- Django (Python) – built-in ORM, CSRF, auth system
- Laravel (PHP) – validation, middleware, Eloquent ORM
- Express.js + Helmet.js (Node.js) – secure headers
- Spring Boot Security (Java)
- ASP.NET Core MVC (C#) – identity framework

7. Example Injection-Free Requirements

✓ Use ORM queries only

✓ Use prepared statements

✓ Input validation applied everywhere

✓ Output encoding on all user-supplied data

✓ CSRF protection enabled

✓ Strict Content Security Policy (CSP) recommended